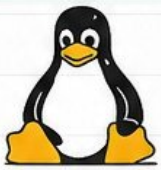
VERIQTA

LINUX FOR COMPLETE BEGINNERS



A PRACTICAL GUIDE FROM ZERO TO CONFIDENT LINUX USER

COMMAND LINE | FILE SYSTEM | USERS & PERMISSIONS | SERVICES & PROCESSES | NETWORKING

- Step-by-step Learning
- Beginner Friendly
- Hands-on Examples
- DevOps Ready
- Real-world Scenarios
- Build Strong Foundation

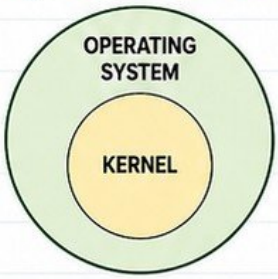
LEARN TODAY. PRACTICE DAILY.
USE LINUX ANYWHERE.

LINUX FROM ZERO

1 WHAT LINUX ACTUALLY IS

- Linux is an open-source operating system.
- It is the most popular OS for servers, cloud, DevOps and engineering work.
- It is stable, secure, fast and highly customizable.

2 LINUX KERNEL VS OPERATING SYSTEM



- **Kernel:** The core part of Linux. It talks to hardware (CPU, memory, storage, devices). It manages system resources.
- **Operating System:** Everything around the kernel. Includes system tools, libraries, commands, applications and utilities.

3 LINUX DISTRIBUTIONS (DISTROS)

- Many distributions use the Linux kernel.
- Each distro packages tools and software differently.

UBUNTU	RHEL	DEBIAN	ROCKY LINUX
User friendly, great for beginners	Enterprise grade, stability and support	Stable, reliable, community driven	Enterprise compatible with RHEL

All are powerful and widely used in real world engineering environments.

4 SERVER LINUX VS DESKTOP LINUX

SERVER LINUX	DESKTOP LINUX
• Runs without UI (GUI). • Optimized for performance, security and uptime. • Used in data centers, cloud and production.	• Has graphical interface (GUI). • Designed for end users. • Used for daily tasks like browsers, office apps, etc.

5 WHY DEVOPS ENGINEERS USE LINUX

- Most servers & cloud run Linux.
- Automation, scripting and tooling work best on Linux.
- Better security and control.
- Essential for DevOps, SRE, Cloud & Platform Engineering.

6 TERMINAL VS SHELL

>_	**Terminal:** The application you open to interact with Linux. It is the window.
$_	**Shell:** The program inside the terminal that interprets and executes your commands. Example: bash, zsh, sh

7 YOUR FIRST LINUX COMMANDS (PRACTICAL)

COMMAND	WHAT IT DOES	EXAMPLE	SAMPLE OUTPUT (MAY VARY)
whoami	Shows the current user	whoami	ubuntu
hostname	Shows the system hostname	hostname	ip-172-31-20-15
uname -a	Shows system & kernel info	uname -a	Linux ip-172-31-20-15 5.15.0-101-generic #112~20.04.1-Ubuntu x86_64 GNU/Linux
date	Shows current date and time	date	Sat May 11 10:30:45 UTC 2024
uptime	Shows how long the system has been running	uptime	10:30:45 up 2 days, 3:15, 1 user, load average: 0.08, 0.07, 0.05

PRACTICE TIP:
Open your terminal and run each command. This builds your confidence and familiarity.

GOAL:
Understand Linux basics and feel comfortable in the terminal.

VERIQTA

UNDERSTANDING THE LINUX FILESYSTEM

THE LINUX FILESYSTEM IS ORGANIZED IN A TREE-LIKE STRUCTURE STARTING FROM THE ROOT DIRECTORY /

DIRECTORY	PURPOSE	EXAMPLE CONTENT
/	Root of the filesystem. Everything starts here.	All directories and files
/home	Home directories for regular users.	/home/user1 /home/user2
/etc	System configuration files.	passwd, hosts, ssh, nginx
/var	Variable data that changes.	logs, spool, cache, databases
/tmp	Temporary files. Usually cleared on reboot.	Temporary files from apps
/usr	User programs and applications.	bin, sbin, lib, share, local
/opt	Optional add-on software packages.	Third-party applications
/bin	Essential user binaries (commands).	ls, cp, mv, cat, bash
/sbin	System binaries for administration.	fdisk, reboot, ifconfig, mkfs
/root	Home directory for the root (admin) user.	Root's personal files

ABSOLUTE VS RELATIVE PATHS

ABSOLUTE PATH

Starts from the root directory / and shows the full location.

Example: /home/user/docs/file.txt

RELATIVE PATH

Starts from your current location.

Example: docs/file.txt

UNDERSTANDING . AND ..

. (DOT)
Refers to the current directory.
Example: ./script.sh

.. (DOT DOT)
Refers to the parent directory.
Example: ../ (go up one level)

QUICK TIP
Linux is case-sensitive. /Home and /home are different directories.

PRACTICAL: NAVIGATE THROUGH A REAL LINUX FILESYSTEM

PRACTICE STEPS

1	pwd	Show your current location	
2	ls -l /	List the root directory	
3	cd /home	Enter the /home directory	
4	ls	List contents	
5	cd ..		Go up one level
6	cd /etc		Go to /etc and explore
7	cd /var/log		Explore log files
8	cd ~		Go to your home directory
9	cd -		Go back to previous directory

GOAL
Understand where important files live so you can work confidently in Linux.

KEY TAKEAWAY
The filesystem hierarchy is the foundation of Linux. Learn it once, use it every day.

VERIQTA

MOVING AROUND LINUX

ESSENTIAL COMMANDS TO NAVIGATE LIKE A PRO

pwd

Print working directory.
Shows your current location.

EXAMPLE

```
$ pwd
/home/user
```

ls

List files and directories.
Shows the contents of your current location.

EXAMPLE

```
$ ls
docs  downloads  file.txt
logs  music      scripts
```

cd

Change directory.
Move into another directory.

EXAMPLE

```
$ cd /var/log
$ pwd
/var/log
```

clear

Clear the terminal screen.
Makes your screen clean and easy to read.

EXAMPLE

```
$ clear
(screen cleared)
```

history

Show previously executed commands.
View or reuse past commands.

EXAMPLE

```
$ history
1  ls -la
2  cd /etc
3  pwd
```

man

View the manual page for a command.
Detailed documentation for any command.

EXAMPLE

```
$ man ls
(manual page
appears)
```

Tab Completion

TAB

Press Tab to auto-complete commands, filenames, and directories.
Saves time and reduces typing errors.

EXAMPLE

```
$ cd Doc<Tab>
$ cd Documents/
```

Hidden Files

Files starting with . are hidden.
Use -a option with ls to see them.

EXAMPLE

```
$ ls -a
.   ..   .bashrc  .config
.git  file.txt  docs
```

USEFUL ls OPTIONS

OPTION	WHAT IT DOES	EXAMPLE	OUTPUT (MAY VARY)
ls -l	Long listing format	$ ls -l	Detailed info (perm, owner, size, date)
ls -a	Show hidden files	$ ls -a	Shows files starting with .
ls -la	Long format + hidden files	$ ls -la	Detailed list including hidden files
ls -lh	Human readable file sizes	$ ls -lh	Shows sizes like 1.2K, 3M
ls -lt	Sorted by modification time	$ ls -lt	Newest files first
ls -R	Recursive listing	$ ls -R	List all files in subdirectories

READING COMMAND HELP

--help	Quick help for most commands.	$ ls --help
man	Full manual page.	$ man ls
info	Detailed info (if available).	$ info ls

TIP:
Whenever you forget a command, use --help or man before guessing.

PRACTICAL: NAVIGATE BETWEEN DIRECTORIES WITHOUT USING A GUI

1. Start at your home directory. — `$ pwd /home/user`
2. List files and folders. — `$ ls`
3. Move into a directory. — `$ cd folder_name`
4. Move deeper into subfolders. — `$ cd subfolder`
5. Move back to parent directory. — `$ cd ..`
6. Go back to your home. — `$ cd ~`

PRACTICE DAILY: The more you use the terminal, the faster and more confident you become.

VERIQTA

CREATING, COPYING, MOVING, AND DELETING

MASTER THE BASICS OF FILE AND DIRECTORY MANAGEMENT

touch

Create an empty file or update file timestamp.

EXAMPLE

```
$ touch file.txt
$ touch notes.md
```

rm

Remove (delete) files or directories.

EXAMPLE

```
$ rm file.txt
$ rm -r old_project/
```

mkdir

Create a new directory (folder).

EXAMPLE

```
$ mkdir project
$ mkdir src logs
```

rmdir

Remove an empty directory.

EXAMPLE

```
$ rmdir empty_folder
```

cp

Copy files or directories from one place to another.

EXAMPLE

```
$ cp file.txt backup.txt
$ cp -r project/
  project_backup/
```

RECURSIVE OPERATIONS

Use `-r` (recursive) to operate on directories and their contents.

EXAMPLE

```
$ cp -r src/ backup_src/
$ rm -r logs/
```

mv

Move or rename files and directories.

EXAMPLE

```
$ mv file.txt newname.txt
$ mv project/ old_project/
```

WILDCARDS

Use * to match multiple characters.
? matches a single character.

EXAMPLE

```
$ rm *.log
$ rm file?.txt docs/
```

SAFE DELETION HABITS

- Always preview before deleting.
  `$ ls -l *.log`
- Use `rm -i` to confirm each deletion.
  `$ rm -i file.txt`
- Avoid `rm -rf /` (very dangerous).
  `$ rm -rf /  # NEVER DO THIS`
- Double-check paths and filenames.
  `$ pwd`

PRACTICAL: CREATE, ORGANIZE, COPY, AND CLEAN UP A PROJECT

1. Create project directory.
   ```
   $ mkdir my_project
   $ cd my_project
   ```
2. Create subdirectories.
   ```
   $ mkdir src docs logs
   ```
3. Create some files.
   ```
   $ touch src/app.py
   $ touch docs/readme.md
   $ touch logs/app.log
   ```
4. List project structure.
   ```
   $ ls -R
   ```
5. Copy a file.
   ```
   $ cp docs/readme.md
     docs/readme_backup.md
   ```
6. Move/rename a file.
   ```
   $ mv src/app.py
     src/main.py
   ```
7. Copy entire directory.
   ```
   $ cp -r src/ src_backup/
   ```
8. Delete a file.
   ```
   $ rm logs/app.log
   ```
9. Remove empty directory.
   ```
   $ rmdir logs
   ```
10. Final project structure.
    ```
    my_project/
    ├── docs/
    │   └── readme.md
    ├── src/
    │   ├── main.py
    │   └── main_backup.py
    └── src_backup/
    ```

KEY TAKEAWAY: Once you master these commands, you can create, organize, manage, and clean up files and directories confidently from the terminal.

VERIQTA

READING AND EDITING FILES

VIEW, INSPECT, AND EDIT FILES LIKE A PRO

cat

Display the entire contents of a file.

EXAMPLE

```
$ cat file.txt
```

less

View file content one screen at a time. Navigate easily.

EXAMPLE

```
$ less largefile.log
Keys: Space (down)
b (up)     q (quit)
```

head

Show the first 10 lines of a file (default).

EXAMPLE

```
$ head file.txt
$ head -n 20 file.txt
```

tail

Show the last 10 lines of a file (default).

EXAMPLE

```
$ tail file.txt
$ tail -n 50 file.txt
```

nano

Simple and user-friendly text editor for beginners.

EXAMPLE

```
$ nano config.conf
Keys: Ctrl+O (save)
Ctrl+X (exit)
```

introduction to vim

Powerful text editor for advanced users.
Three modes:
Insert | Normal | Command

EXAMPLE

```
$ vim file.txt
Keys:
i (insert)  Esc (normal)
:w (save)   :q (quit)
:wq (save  & quit)
```

wc

Count lines, words, and characters in a file.

EXAMPLE

```
$ wc file.txt
$ wc -l file.txt
  (lines only)
```

viewing large files

Use less to open large files without loading everything into memory.

EXAMPLE

```
$ less very_large.log
Search: /error (Enter)
Next: n      Prev: N
```

following changing files with tail -f

Monitor logs in real time.
New lines appear instantly.

EXAMPLE

```
$ tail -f app.log
Press Ctrl+C to stop.
```

COMMON tail OPTIONS

-f	Follow the file continuously (live logs)
-n	Show last n lines
-F	Follow by name (handle log rotation)

TEXT EDITOR QUICK REFERENCE

NANO ESSENTIALS

Open file	$ nano filename
Save file	Ctrl + O, then Enter
Exit editor	Ctrl + X
Cut line	Ctrl + K
Paste line	Ctrl + U
Search	Ctrl + W

TIP

Nano is perfect for beginners. Use it for config files and quick edits.

VIM ESSENTIALS

Open file	$ vim filename
Insert mode	Press i
Save file	:w
Save & exit	:wq
Quit (without save):	:q!
Search	/text (Enter)
Exit insert mode	Press Esc

TIP

Vim is powerful at first, but very efficient once learned.

PRACTICAL: CREATE, EDIT, AND INSPECT FILES

1. Create a directory for the lab.

```
$ mkdir file_lab
$ cd file_lab
```

2. Create a configuration file with nano.

```
$ nano app.conf
# add some content
Ctrl+O (save)
Ctrl+X (exit)
```

3. View the file content using cat.

```
$ cat app.conf
```

4. Check first and last lines of a log file.

```
$ head app.log
$ tail app.log
```

5. Monitor a log file in real time.

```
$ tail -f app.log
```

6. Count lines, words, and characters.

```
$ wc app.log
```

KEY TAKEAWAY: Master these commands and you can read, inspect, monitor, and edit any file in Linux confidently. These skills are essential for DevOps and system engineering.

VERIQTA

LINUX USERS AND GROUPS

MANAGE ACCESS, PERMISSIONS, AND OWNERSHIP

KEY CONCEPTS

What a Linux user is
A user is an identity that can log in and run processes.

Root user
The superuser with UID 0. Has unlimited access.

Regular users
Human users with limited permissions.

System users
Created by the system or applications. Usually cannot login interactively.

UID and GID
UID (User ID) uniquely identifies a user. GID (Group ID) identifies a group.

IMPORTANT FILES

/etc/passwd

Stores user account information.

```
$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
alice:x:1001:1001:Alice:/home/alice:/bin/bash
bob:x:1002:1002:Bob:/home/bob:/bin/bash
```

Format:
username:x:UID:GID:comment:home:shell

/etc/group

Stores group information.

```
$ cat /etc/group
root:x:0:
developers:x:1001:alice,bob
ops:x:1002:charlie
```

Format:
groupname:x:GID:user1,user2,user3

ESSENTIAL COMMANDS

	EXAMPLE
useradd Create a new user.	`$ sudo useradd alice`
usermod Modify user properties.	`$ sudo usermod -aG developers alice`
userdel Delete a user account.	`$ sudo userdel bob`
groupadd Create a new group.	`$ sudo groupadd developers`
id Show user and group IDs.	`$ id alice`
groups Show groups a user belongs to.	`$ groups alice`

USER AND GROUP INFORMATION AT A GLANCE

TYPE	DESCRIPTION	EXAMPLE	UID	GID	LOGIN ABILITY	HOME DIRECTORY
Root User	Superuser with all privileges	root	0	0	Yes	/root
Regular User	Human user for daily tasks	alice	1001+	1001+	Yes	/home/alice
System User	Used by system services	www-data	1–999	1–999	No	/nonexistent

COMMON usermod OPTIONS

-aG group	Add user to supplementary group
-G group	Set primary group
-l newname	Change username
-d /path	Change home directory
-s /shell	Change login shell

CHECK USER DETAILS

```
$ id alice
uid=1001(alice) gid=1001(alice)
groups=1001(alice),1001(developers)

$ groups alice
alice : alice developers
```

BEST PRACTICES

- Use regular users for daily work.
- Give minimum necessary privileges.
- Never share root credentials.
- Audit user access regularly.
- Remove unused accounts.

PRACTICAL: CREATE A USER AND ASSIGN TO A GROUP

1. Create a group. → `$ sudo groupadd developers`
2. Create a new user. → `$ sudo useradd -m -s /bin/bash alice`
3. Set a password for the user. → `$ sudo passwd alice`
4. Add user to the group. → `$ sudo usermod -aG developers alice`
5. Verify user details with id. → `$ id alice`
6. Check group membership. → `$ groups alice`
7. List all users. → `$ cat /etc/passwd`

KEY TAKEAWAY: Users and groups control who can access what in Linux.
Understanding them is fundamental for system security and administration.

VERIQTA

FILE OWNERSHIP AND PERMISSIONS

CONTROL WHO CAN READ, WRITE, AND EXECUTE FILES

1. PERMISSION BASICS

READ (r)
View file contents or list directory contents.

WRITE (w)
Modify file contents or create/delete files in a directory.

EXECUTE (x)
Run a file or enter (cd into) a directory.

2. USER, GROUP, OTHERS

USER (u)
The owner of the file or directory.

GROUP (g)
The group that owns the file or directory.

OTHERS (o)
Everyone else on the system.

3. UNDERSTANDING RWX

EXAMPLE: `-rwxr-x---`

rwx	r-x	---
USER	GROUP	OTHERS

MEANING:

- Owner: read, write, execute (rwx)
- Group: read, execute (r-x)
- Others: no permission (---)

4. PERMISSION NUMERIC VALUES

PERMISSION	SYMBOL	NUMERIC VALUE
Read	r	4
Write	w	2
Execute	x	1
None	-	0

HOW NUMERIC PERMISSIONS WORK

Add the values:

r (4) + w (2) + x (1) = 7 (rwx)
r (4) + x (1) = 5 (r-x)
r (4) = 4 (r--)
w (2) + x (1) = 3 (-wx)
w (2) = 2 (-w-)
x (1) = 1 (--x)
--- (0) = 0 (---)

COMMON NUMERIC PERMISSIONS

NUMERIC	RWX	MEANING
755	rwxr-xr-x	Owner full, Group read+execute, Others read+execute
644	rw-r--r--	Owner read+write, Group read, Others read
600	rw-------	Owner read+write only
664	rw-rw-r--	Owner read+write, Group read+write, Others read
700	rwx------	Owner full only
777	rwxrwxrwx	Everyone full (DANGEROUS)

5. CHMOD

Change file or directory permissions.
Syntax: chmod [options] mode file
Examples:

```
$ chmod 755 script.sh
$ chmod -R 644 data/
```

6. CHOWN

Change ownership (user and group).
Syntax: chown [user[]:group] file
Examples:

```
$ chown alice file.txt
$ chown alice:developers file.txt
```

7. CHGRP

Change group ownership.
Syntax: chgrp [group] file
Examples:

```
$ chgrp developers file.txt
$ chgrp -R developers project/
```

8. WHY CHMOD 777 IS BAD

- Gives full access to everyone.
- Anyone can read, modify,
- or delete your files.
- Creates security risks.
- Attackers can exploit writable files.
- Use the principle of least privilege.

9. PERMISSION DENIED TROUBLESHOOTING

1. Check file owner: $ ls -l file
2. Check permissions: $ ls -l file
3. Check your user: $ whoami
4. Check your groups: $ groups
5. Fix permissions: $ chmod ...
6. Fix ownership: $ chown ...
7. Try again.

10. QUICK REFERENCE

View detailed permissions
`$ ls -l`
Change permissions
`$ chmod 755 file`
Change owner
`$ chown user file`
Change group
`$ chgrp group file`

PRACTICAL: DIAGNOSE AND REPAIR A FILE PERMISSION PROBLEM

1. Create a project directory and file.

```
$ mkdir lab
$ cd lab
$ touch app.sh
```

2. Give wrong permissions.

```
$ chmod 000 app.sh
$ ls -l app.sh
----------
app.sh
```

3. Try to run the file.

```
$ ./app.sh
bash: ./app.sh:
Permission denied
```

4. Check ownership and permissions.

```
$ ls -l app.sh
---------- 1 alice
alice 0 May 24 10:00
app.sh
```

5. Fix permissions (owner full).

```
$ chmod 700 app.sh
$ ls -l app.sh
-rwx------ 1 alice
alice 0 May 24 10:00
app.sh
```

6. Run again (works now).

```
$ ./app.sh
Hello from
app.sh
```

7. Change group ownership.

```
$ usermod -aG devs bob
$ ls -l app.sh
-rwx------ 1 alice
devs 0 May 24 10:00
app.sh
```

KEY TAKEAWAY: Properly managing file ownership and permissions keeps your system secure, stable, and predictable. Grant the minimum access required—nothing more.

VERIQTA

SUDO, ROOT, AND PRIVILEGE

CONTROL. SECURITY. RESPONSIBILITY.

1. WHAT ROOT CAN DO

- Install or remove software
- Modify system files
- Manage users and groups
- Start/stop system services
- Change network settings
- Access all files and data
- Bypass normal restrictions

ROOT HAS UNLIMITED POWER. USE CAREFULLY.

2. SUDO

Run a command as another user (usually root).

- Allows limited admin access
- Uses your own password (if configured)
- Safer than using root directly

```
$ sudo command
$ sudo systemctl restart nginx
$ sudo apt update
```

3. SU

Switch user. Commonly used to become root.

- Becomes the target user
- Usually requires the target user's password
- Provides full access of that user

```
$ su -
Password:
# now you are root
```

4. PRIVILEGE ESCALATION

Moving from a normal user to a higher privilege (like root).

- Legitimate: sudo, su
- Illegitimate: exploits, misconfigurations
- Always follow least privilege principle

5. /ETC/SUDOERS

File that controls who can run what as root using sudo.

- Located at: /etc/sudoers
- Direct editing can break your system!

NEVER EDIT THIS FILE WITH A NORMAL TEXT EDITOR.

6. VISUDO

Safe way to edit /etc/sudoers.

- Syntax checking
- Prevents lockout
- Use this command always

```
$ sudo visudo
```

7. LEAST PRIVILEGE

Give users only the access they need.

- Reduces security risks
- Limits damage from mistakes or attacks
- Essential for secure systems

8. WHY RUNNING EVERYTHING AS ROOT IS DANGEROUS

- Any mistake can break the system
- Malware gets full control
- No accountability
- Harder to audit actions
- Violates security best practices

9. QUICK COMMAND REFERENCE

whoami	Show current user
sudo -l	List sudo permissions
sudo -v	Refresh sudo token
su -	Switch to root user
id	Show user and groups
groups	Show your groups

10. PRACTICAL: ALLOW A USER TO PERFORM AN ADMINISTRATIVE TASK SAFELY

GOAL: Allow user "devops" to restart nginx without giving full root access.

1. Create a user (if not exists)

```
$ sudo useradd devops
$ sudo passwd devops
```

2. Allow user to run systemctl restart nginx

Run visudo:

```
$ sudo visudo
```

3. Add this line at the end of the file

```
devops ALL=(root)
NOPASSWD:
/bin/systemctl restart nginx
```

4. Save and exit visudo

File saved safely with correct syntax.

5. Test as devops user

```
$ sudo -u devops -i
$ sudo systemctl
restart nginx
```

6. Verify it works

```
$ systemctl status
nginx
```

SUCCESS!
User "devops" can restart nginx using sudo, but cannot run other administrative commands.

IMPORTANT
Grant only the specific command needed.
Avoid giving NOPASSWD access unless necessary and trusted.

PAGE 9

VERIQTA

FINDING FILES AND INFORMATION

THE RIGHT COMMAND SAVES TIME.

1. FIND

Search for files and directories in real time. Very powerful.

SYNTAX

find [path] [options] [expression]

EXAMPLE

```
$ find /etc -name my.conf
```

2. LOCATE

Search using a prebuilt database. Very fast.

SYNTAX

locate [filename]

EXAMPLE

```
$ locate nginx.conf
```

Database updated by: updatedb

3. WHICH

Show the full path of a command.

SYNTAX

which [command]

EXAMPLE

```
$ which nginx
```

4. WHEREIS

Locate binaries, source code and man pages.

SYNTAX

whereis [name]

EXAMPLE

```
$ whereis python3
```

5. GREP

Search text patterns inside files.

SYNTAX

grep [options] "pattern" [file]

EXAMPLE

```
$ grep "Listen" /etc/nginx/nginx.conf
```

6. COMMON SEARCH TYPES WITH FIND

A. RECURSIVE SEARCH

Search in current directory and all subdirectories.

```
$ find . -name "*.log"
```

B. SEARCH BY FILENAME

Find a file by its name.

```
$ find / -name nginx.conf 2>/dev/null
```

C. SEARCH BY FILE TYPE

Find files of a specific type.
f = file, d = directory

```
$ find /etc -type f -name "*.conf"
$ find /var -type d -name "cache"
```

D. SEARCH BY SIZE

Find files based on size.
+100M = larger than 100MB
-10M = smaller than 10MB

```
$ find . -type f -size +100M
$ find . -type f -size -10M
```

E. SEARCH BY TIME

Find files modified in the last 1 day.

```
$ find . -type f -mtime -1
```

7. COMBINING FIND AND GREP (POWERFUL!)

Find files and search inside them.

```
$ find /etc -type f -name "*.conf" -exec grep -H "server" {} \;
```
→ Find .conf files and search for "server" inside them.

```
$ grep -R "root" /etc/
```
→ Search recursively for "root" inside /etc directory.

```
$ find /var/log -type f -exec grep -H "error" {} \;
```
→ Find log files containing the word "error".

8. PRACTICAL: FIND A CONFIGURATION FILE AND LOCATE A SPECIFIC VALUE

1. Find the nginx configuration file.
```
$ find /etc -type f -name "nginx.conf" 2>/dev/null
```
2. Verify the file exists.
```
$ ls -l /etc/nginx/nginx.conf
```
3. Search for a specific value inside the file.
```
$ grep -n "listen" /etc/nginx/nginx.conf
```
4. Search recursively in all .conf files for a value.
```
$ grep -R "server_name" /etc/*.conf
```
5. Find files modified in last 1 day.
```
$ find /etc -type f -mtime -1
```

SUCCESS!
You found the file and the required information using the right tools.

TIPS
- Use find for accurate real-time search.
- Use locate for quick results (after running updatedb).
- Use grep to search inside files.
- Combine commands for powerful results.

REMEMBER
- Searching whole system (/) can be slow.
- Use 2>/dev/null to hide permission denied errors.

VERIQTA

PIPES AND REDIRECTION

CONTROL WHERE COMMAND INPUT AND OUTPUT GO

1. STANDARD INPUT (STDIN)

Data that a command receives as input.

Usually from keyboard or a file.

```
$ command < file.txt
```

2. STANDARD OUTPUT (STDOUT)

Normal output of a command.

Displayed on screen by default.

```
$ command > output.txt
```

3. STANDARD ERROR (STDERR)

Error messages from a command.

Displayed on screen by default.

```
$ command 2> error.log
```

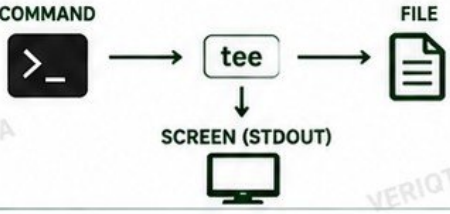
4. REDIRECTION OPERATORS

OPERATOR	NAME	DESCRIPTION	SYNTAX EXAMPLE	EXAMPLE USE
>	Redirect Output	Redirect standard output (overwrite).	`$ command > file.txt`	Save output to file.txt (overwrite if file exists).
>>	Append Output	Redirect standard output (append).	`$ command >> file.txt`	Append output to file.txt (create if not exists).
<	Redirect Input	Use file as standard input instead of keyboard.	`$ command < file.txt`	Read input from file.txt instead of typing.
2>	Redirect Error	Redirect standard error (overwrite).	`$ command 2> error.log`	Save errors to error.log (overwrite if exists).
2>>	Append Error	Redirect standard error (append).	`$ command 2>> error.log`	Append errors to error.log.
|	Pipe	Send output of one command as input to another.	`$ command1 | command2`	Use output of command1 as input to command2.
tee	Tee	Read from input and write to both output and file.	`$ command | tee file.txt`	Show output on screen and save to file.txt.

5. WHY PIPELINES MATTER

- Chain multiple commands together.
- Filter and transform data quickly.
- Reduce intermediate files.
- Automate complex tasks easily.
- Essential skill for DevOps, SRE and system administrators.

6. TEE COMMAND (EXAMPLE)



tee duplicates the input:

- One copy to screen
- One copy to file

```
$ ls -l | tee listing.txt
```

7. PRACTICAL: TAKE COMMAND OUTPUT, FILTER IT, AND SAVE TO A FILE

1. List all running processes.
   ```
   $ ps aux
   ```
2. Filter lines containing "nginx".
   ```
   $ ps aux | grep nginx
   ```
3. Show output on screen and save to a file.
   ```
   $ ps aux | grep nginx | tee nginx_processes.txt
   ```
4. Count the number of nginx processes.
   ```
   $ ps aux | grep nginx | wc -l
   ```
5. Save only errors to a log file.
   ```
   $ some_command 2> errors.log
   ```
6. Append output to log file.
   ```
   $ echo "Done" >> process.log
   ```

QUICK REFERENCE

>	redirect output (overwrite)
>>	redirect output (append)
<	redirect input
2>	redirect error (overwrite)
2>>	redirect error (append)
|	pipe
tee	duplicate to file and screen

TIPS

- Use quotes when searching for text with spaces.
- Combine pipes with grep, awk, sort, uniq, wc, head, tail for powerful text processing.
- Always check your redirections before running important commands.

KEY TAKEAWAY

Pipes and redirection let you control the flow of data between commands, files, and the screen—making you more efficient and powerful on the Linux CLI.

PROCESSES AND JOBS

UNDERSTAND, MONITOR AND CONTROL RUNNING TASKS IN LINUX

1 WHAT IS A PROCESS?

- A process is an instance of a running program.
- Every process has a unique Process ID (PID).
- The parent process that started it has a Parent Process ID (PPID).

2 PID AND PPID

- PID: Unique ID of the process.
- PPID: ID of the parent process.
- All processes (except init/systemd) have a parent.

Parent Process (PPID) → Child Process (PID)

3 FOREGROUND VS BACKGROUND

FOREGROUND

Runs in the current terminal.
You can see the output.
You must wait for it to finish.

BACKGROUND

Runs behind the scenes.
Does not block your terminal.
You can continue working.

4 KEY COMMANDS

COMMAND	PURPOSE	EXAMPLE
ps	Show running processes snapshot	ps aux
top	Interactive real-time process monitor	top
pgrep	Search for processes by name	pgrep nginx
kill	Send signal to a process using PID	kill 1234
pkill	Send signal to processes by name	pkill nginx
jobs	List background and stopped jobs	jobs
bg	Resume a stopped job in background	bg %1
fg	Bring a background job to foreground	fg %1
&	Run a command in background	sleep 100 &

5 SIGNALS (COMMON ONES)

SIGHUP 1	SIGINT 2	SIGTERM 15	SIGKILL 9	SIGSTOP 19	SIGCONT 18
Reload / Reopen config	Ctrl + C Interrupt (terminate)	Graceful terminate	Force kill (cannot be ignored)	Stop / Pause a process	Continue a stopped process

TIP

Use SIGTERM first.
Use SIGKILL only when necessary.

6 PRACTICAL: IDENTIFY A RUNNING PROCESS AND STOP IT SAFELY

1 LIST PROCESSES	2 FIND PID	3 VERIFY PROCESS	4 GRACEFUL STOP	5 FORCE STOP	6 CONFIRM
Use ps aux or top to find the process.	Note the PID of the process.	Confirm it is the correct process.	Try SIGTERM first (graceful stop).	If still running, use SIGKILL.	Check again to make sure it stopped.
ps aux	ps aux | grep nginx	ps -p <PID>	kill <PID>	kill -9 <PID>	ps aux | grep <name>

BEST PRACTICE
Always try a graceful stop (SIGTERM) first. Use SIGKILL (-9) only when necessary.

WARNING
Killing critical system processes can make your system unstable.

GOAL
Understand how processes work and control them confidently.

VERIQTA

LINUX SERVICES WITH SYSTEMD

MANAGE, CONTROL AND AUTOMATE SERVICES ON LINUX

1 WHAT IS A SERVICE?

- A service is a background program that starts, runs and continues working to provide a function.
- Examples: web server, database, SSH server, logging, etc.

2 SYSTEMD

- **systemd** is the init system and service manager for most modern Linux distributions.
- It starts services at boot, supervises them, restarts on failure and manages dependencies.

3 UNITS

- systemd manages everything as "units".
- The most common unit type is service (.service).
- Other unit types: socket, timer, mount, target, etc.

4 BASIC SYSTEMCTL COMMANDS

COMMAND	PURPOSE	EXAMPLE	DESCRIPTION
systemctl status <service>	Check status of a service	systemctl status nginx	• Shows if the service is active, inactive, failed, etc. • Displays logs and last few messages.
systemctl start <service>	Start a service	systemctl start nginx	Starts the service immediately.
systemctl stop <service>	Stop a running service	systemctl stop nginx	Stops the service immediately.
systemctl restart <service>	Restart a service	systemctl restart nginx	Stops and starts the service again.
systemctl enable <service>	Enable service at boot	systemctl enable nginx	• Ensures the service starts automatically when the system boots. • Creates a symlink in system targets.
systemctl disable <service>	Disable service at boot	systemctl disable nginx	• Prevents the service from starting automatically at boot. • Removes the symlink.

5 SERVICE STARTUP AT BOOT

- When enabled, the service starts automatically during system boot.
- Check if enabled:

```
systemctl is-enabled <service>
```

- Outputs: enabled / disabled / static

6 FAILED SERVICES

- If a service fails, systemd will show it as "failed".
- Check failed services:

```
systemctl --failed
```

- To see details and logs:

```
systemctl status <service>
```

7 COMMON SERVICE STATES

active (running)	Service is running.
active (exited)	Service ran and exited.
inactive (dead)	Service is stopped.
failed	Service failed to start.
activating	Service is starting.
deactivating	Service is stopping.

8 PRACTICAL: START, STOP, ENABLE AND TROUBLESHOOT A SERVICE

1 CHECK STATUS
Check current status of the service.
`systemctl status <service>`

2 START SERVICE
Start the service.
`systemctl start <service>`

3 VERIFY RUNNING
Verify it is running.
`systemctl status <service>`

4 ENABLE AT BOOT
Enable service to start automatically.
`systemctl enable <service>`

5 STOP SERVICE
Stop the service.
`systemctl stop <service>`

6 TROUBLESHOOT
If failed, check logs and error details.
`systemctl status <service>`

TIPS
- Always check status before making changes.
- Use `systemctl status <service>` to read logs and errors.

BEST PRACTICE
Enable only necessary services. Keep your system clean and secure.

WARNING
Stopping critical services can cause your system or applications to fail.

PAGE 13

VERIQTA

INSTALLING AND MANAGING SOFTWARE

MANAGE PACKAGES, UPDATES AND DEPENDENCIES IN LINUX

1 WHAT IS A PACKAGE?

- A package is a pre-compiled software bundle.
- Contains the program and everything it needs to run.
- Includes files, libraries, dependencies and configuration.

2 PACKAGE REPOSITORIES

- Repositories store packages.
- Your system connects to repos to download and update software.
- Official repos are tested and reliable.

3 PACKAGE MANAGERS

DEBIAN / UBUNTU
apt
Advanced Packaging Tool

RHEL / ROCKY / ALMALINUX
dnf
Dandified Yum

RHEL / CENTOS 7
yum
Yellowdog Updater Modified

4 WHAT THEY DO

- Install new software
- Update existing software
- Remove software
- Resolve dependencies automatically
- Keep system consistent and secure
- Save time and prevent manual dependency errors

5 WHY PRODUCTION UPDATES REQUIRE CARE

- Updates can introduce breaking changes.
- May affect application compatibility.
- Always test updates before applying.
- Use backups and maintenance windows.
- Read release notes.
- Update one system at a time.

6 ESSENTIAL PACKAGE MANAGEMENT COMMANDS

TASK	DEBIAN / UBUNTU (apt)		RHEL / ROCKY / ALMALINUX (dnf)		RHEL / CENTOS 7 (yum)	
	COMMAND	EXAMPLE	COMMAND	EXAMPLE	COMMAND	EXAMPLE
Update package list	sudo apt update	sudo apt update	sudo dnf makecache	sudo dnf makecache	sudo yum check-update	sudo yum check-update
Install package	sudo apt install <pkg>	sudo apt install nginx	sudo dnf install <pkg>	sudo dnf install nginx	sudo yum install <pkg>	sudo yum install nginx
Update packages	sudo apt upgrade	sudo apt upgrade	sudo dnf upgrade	sudo dnf upgrade	sudo yum update	sudo yum update
Remove package	sudo apt remove <pkg>	sudo apt remove nginx	sudo dnf remove <pkg>	sudo dnf remove nginx	sudo yum remove <pkg>	sudo yum remove nginx
Check installed packages	dpkg -l	dpkg -l | less	dnf list installed	dnf list installed | less	yum list installed	yum list installed | less
Search for package info	apt search <pkg>	apt search nginx	dnf search <pkg>	dnf search nginx	yum search <pkg>	yum search nginx

NOTE: Always use **sudo** (administrative privileges) for package management commands.

7 PRACTICAL: INSTALL AND VERIFY A WEB SERVER PACKAGE (NGINX)

1 UPDATE REPO LIST
Update the package index first.
`sudo apt update` # or `sudo dnf makecache`

2 INSTALL NGINX
Install the nginx web server.
`sudo apt install nginx` # or `sudo dnf install nginx`

3 START SERVICE
Start nginx immediately.
`sudo systemctl start nginx`

4 VERIFY STATUS
Check if nginx is running.
`systemctl status nginx`

5 TEST IN BROWSER
Open your server IP in a browser.
`http://<your-server-ip>`
You should see the nginx welcome page.

6 ENABLE AT BOOT
Enable nginx to start automatically.
`sudo systemctl enable nginx`

- If service fails: systemctl status nginx
- Check logs: journalctl -u nginx
- Verify port: ss -tulpn | grep :80
- Check configuration: nginx -t
- Restart after changes: systemctl restart nginx
- Firewall blocking? Allow port 80.

COMMON NGINX PORT
80/tcp
HTTP (Web Traffic)

BEST PRACTICE
- Keep your system updated but tested.
- Read changelogs before major updates.
- Update during maintenance windows.

SECURITY REMINDER
- Only install what you need.
- Remove unused packages.
- Keep your system clean and secure.

WARNING
Unplanned updates can break applications. Always test before updating production environments.

VERIQTA

LINUX NETWORKING BASICS

THE ESSENTIAL NETWORKING SKILLS FOR EVERY LINUX ENGINEER

1 IP ADDRESSES

- Every device on a network has an IP address.
- IPv4 example: 192.168.1.10

2 PRIVATE VS PUBLIC IP

- PRIVATE IP (internal use)
  - 10.0.0.0/8
  - 172.16.0.0/12
  - 192.168.0.0/16
- PUBLIC IP (internet reachable)
  - Assigned by ISP or Cloud

3 NETWORK INTERFACES

- Linux uses interfaces to connect to networks.
- Examples: eth0, ens33, enp0s3, ens5

4 PORTS

- Ports identify services on a host.
- Examples:
  - 22 (SSH)
  - 80 (HTTP)
  - 443 (HTTPS)

5 DNS (DOMAIN NAME SYSTEM)

- Converts domain names (e.g. google.com) to IP addresses.
- Without DNS, we must remember IPs.

6 DEFAULT GATEWAY

- The gateway sends traffic to other networks and the internet.
- Usually your router IP address.

7 COMMON TOOLS

ip addr, ip route, ping, curl, ss, hostname, dig

These are your daily networking tools.

QUICK TIP

Most connectivity problems fall into four areas: DNS, Network, Port, or Application.

8 NETWORKING COMMANDS CHEAT SHEET

COMMAND	PURPOSE	EXAMPLE	SAMPLE OUTPUT (MAY VARY)
ip addr	Show IP addresses of all interfaces	ip addr	2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> inet 192.168.1.10/24 brd 192.168.1.255 scope global ens33
ip route	Show routing table	ip route	default via 192.168.1.1 dev ens33 192.168.1.0/24 dev ens33 proto kernel scope link src 192.168.1.10
ping	Test connectivity to a host	ping 8.8.8.8	64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=20.1 ms
curl	Fetch data from a URL	curl -I https://google.com	HTTP/2 200 content-type: text/html; charset=UTF-8
ss	Show open ports and connections	ss -tulnp	Netid State Recv-Q Send-Q Local Address:Port LISTEN 0 128 0.0.0.0:22 LISTEN 0 128 0.0.0.0:80
hostname	Show or set system hostname	hostname	server01
dig	DNS lookup tool	dig google.com	;; ANSWER SECTION: google.com. 142 IN A 142.250.72.206

9 PRACTICAL: DIAGNOSE CONNECTIVITY PROBLEMS

STEP	CHECK	COMMAND	WHAT TO LOOK FOR	IF IT FAILS
1	DNS Resolution	dig example.com	Look for an IP address in the ANSWER SECTION	If no IP: DNS problem (Check DNS server or /etc/resolv.conf)
2	Network Connectivity	ping <IP-address>	Replies from the IP address	If no reply: Network problem (Routing, Firewall, or Host down)
3	Port Reachability	ss -tulnp | grep :<port> (Or from another host) telnet <IP> <port>	Port should be LISTENING or reachable	If connection refused/timeout: Port blocked or Service not listening
4	Application Check	curl -I http://<IP>[:port]	You should get HTTP response (200, 301, 403, etc.)	If no response or HTTP errors: Application problem (config, service)

10 COMMON EXAMPLES

- Ping gateway: ping 192.168.1.1
- Ping internet: ping 8.8.8.8
- Check default route: ip route | grep default
- Check DNS server: cat /etc/resolv.conf
- Check open ports: ss -tulnp
- Get public IP: curl ifconfig.me

TROUBLESHOOTING FLOW

DNS → NETWORK → PORT → APPLICATION

Fix the first failure before moving to the next step.

BEST PRACTICES

- Always test DNS first.
- Then test network reachability.
- Then test the port.
- Finally test the application.
- Document findings and fix the root cause.

SSH AND REMOTE LINUX SERVERS

CONNECT SECURELY, TRANSFER FILES, AND MANAGE REMOTE SERVERS

1 WHAT SSH DOES

- SSH (Secure Shell) provides secure encrypted communication over insecure networks.
- Replaces insecure protocols like Telnet and rlogin.

2 SSH CLIENT AND SERVER

- Client: Your local machine.
- Server: Remote Linux server running SSH daemon (sshd).
- Default SSH port: 22/TCP.

3 SSH USER@SERVER

Basic syntax:

```
ssh user@server_ip
```

- Connects to the server as the specified user.
- Example:

```
ssh ubuntu@192.168.1.10
```

4 PASSWORD AUTHENTICATION

- The simplest method.
- You enter the user password when connecting.
- Less secure than key-based authentication.

5 SSH KEYS

- More secure and recommended.
- Uses asymmetric cryptography.
- No password needed after setup.

6 PUBLIC VS PRIVATE KEYS

PUBLIC KEY

- Stored on the server.
- Shared.

PRIVATE KEY

- Stored on your local machine.
- Never shared.

7 SSH-KEYGEN

- Generates a key pair.
- Default location:

```
~/.ssh/id_rsa
```

- id_rsa → Private key
- id_rsa.pub → Public key

8 AUTHORIZED_KEYS

- The server stores your public key in:

```
~/.ssh/authorized_keys
```

- Allows key-based login.
- Permissions are critical.

9 SCP (SECURE COPY)

- Securely copy files between local and remote systems.
- Uses SSH for encryption.

```
scp file user@server:/path
```

10 SECURE KEY PERMISSIONS

Keep keys private!

```
chmod 600 ~/.ssh/id_rsa
chmod 700 ~/.ssh
chmod 644 ~/.ssh/id_rsa.pub
```

- Wrong permissions can cause SSH to refuse keys.

11 ESSENTIAL SSH COMMANDS

COMMAND	PURPOSE	EXAMPLE	DESCRIPTION
ssh user@server	Connect to remote server	ssh ubuntu@192.168.1.10	Connect using default port 22.
ssh -p PORT user@server	Connect using custom port	ssh -p 2222 ubuntu@server	Use when SSH runs on a custom port.
scp source user@server:/path	Copy file to remote server	scp file.txt ubuntu@server:/home/ubuntu/	Upload file to server.
scp user@server:/path dest	Copy file from remote server	scp ubuntu@server:/etc/hosts ./	Download file from server.
ssh-keygen -t rsa -b 4096	Generate SSH key pair	ssh-keygen -t rsa -b 4096	Recommended: 4096-bit RSA.
ssh-copy-id user@server	Copy public key to server	ssh-copy-id ubuntu@server	Automatically adds key to authorized_keys.
ssh -i ~/.ssh/id_rsa user@server	Connect with specific key	ssh -i ~/.ssh/id_rsa ubuntu@server	Use a different private key.
ssh -v user@server	Verbose connection (debug)	ssh -v ubuntu@server	Shows detailed connection info.

12 PRACTICAL: CONNECT TO A REMOTE SERVER AND TRANSFER A FILE SECURELY

1 GENERATE KEYS

Create a key pair on your local machine.

```
ssh-keygen -t rsa -b 4096
```

2 COPY PUBLIC KEY

Copy your public key to the server.

```
ssh-copy-id user@server
```

3 TEST SSH LOGIN

Login without password.

```
ssh user@server
```

4 CHECK CONNECTION

Verify you are connected.

```
hostname
whoami
```

5 UPLOAD FILE

Transfer file to the server.

```
scp file.txt user@server:/tmp/
```

6 VERIFY FILE

Check file on the server.

```
ls -l /tmp/file.txt
```

7 DOWNLOAD FILE

Download file from server.

```
scp user@server:/tmp/file.txt .
```

8 CLEAN UP

Remove test file (optional).

```
rm /tmp/file.txt
```

13 COMMON SSH FILES AND LOCATIONS

~/.ssh/	SSH directory
~/.ssh/id_rsa	Private key
~/.ssh/id_rsa.pub	Public key
~/.ssh/authorized_keys	Authorized public keys (server)
/etc/ssh/sshd_config	SSH server config

BEST PRACTICES

- Always use key-based authentication.
- Use strong passphrases for your keys.
- Keep private keys private.
- Disable password authentication on servers.
- Change SSH port from 22 (optional).
- Keep your SSH software updated.

TROUBLESHOOTING TIPS

- Permission denied (publickey): Check key permissions.
- Connection refused: SSH service may not be running.
- Timeout: Check network, firewall, or IP address.
- Use ssh -v user@server for detailed logs.
- Ensure port 22 (or custom port) is open.

VERIQTA

STORAGE AND DISK MANAGEMENT

MANAGE DISKS, FILESYSTEMS AND SPACE LIKE A PRO

1 FILESYSTEMS

- A filesystem defines how data is stored and organized on disk.
- Common Linux filesystems: ext4, xfs, btrfs.
- Each filesystem has its own features and limits.

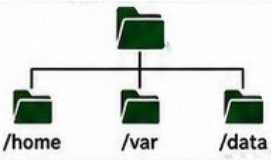
2 DISKS AND PARTITIONS

- A disk (e.g. /dev/sda) can have multiple partitions (e.g. /dev/sda1).
- Each partition can hold a filesystem.

/dev/sda1
/dev/sda2
/dev/sda3

3 MOUNT POINTS

- A mount point is a directory where a filesystem is attached to the directory tree.
- Example: /, /home, /var, /data are mount points.



4 KEY COMMANDS QUICK REFERENCE

COMMAND	PURPOSE	EXAMPLE
df -h	Show filesystem disk space	df -h
du -sh <dir>	Show directory size	du -sh /var
du -ah <dir> | sort -h	Find large files/dirs	du -ah /var | sort -h
lsblk	List block devices	lsblk
mount	Show mounted filesystems	mount
mount <device> <dir>	Mount a filesystem	mount /dev/sdb1 /data
/etc/fstab	Persistent mounts at boot	cat /etc/fstab

5 UNDERSTANDING /etc/fstab

- Controls which filesystems are mounted at boot.
- Format: <device> <mount_point> <filesystem> <options> <dump> <pass>

```
# <device>          <mount point>   <type>   <options>     <dump> <pass>
UUID=1234-5678      /               ext4     defaults      1      1
UUID=abcd-efgh      /home           ext4     defaults      1      2
/dev/sdb1           /data           xfs      defaults      1      2
tmpfs               /tmp            tmpfs    defaults      0      0
```

- Use 'sudo mount -a' to test /etc/fstab without rebooting.
- Bad fstab entries can prevent the system from booting.

6 DISK-SPACE TROUBLESHOOTING

- Run out of space in: /, /var, /home, /tmp, or /data.
- Find what is using the most space.
- Clean old logs, caches, temporary files.
- Move or archive data if needed.

7 FINDING LARGE FILES AND DIRECTORIES

Find what is consuming the most space.

```
# Top 10 largest files under /var
sudo find /var -type f -exec du -h {} + | sort -rh | head -10

# Top 10 largest directories under /var
sudo du -ah /var 2>/dev/null | sort -rh | head -10
```

TIP Check log files, old application data, backups, docker data and caches.

8 COMMAND EXAMPLES

▶ Check disk usage of all filesystems

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   42G  6.5G  87% /
tmpfs           1.9G  1.2M  1.9G   1% /run
/dev/sda2       100G   20G   75G  22% /home
```

▶ Find size of /var directory

```
$ du -sh /var
14G    /var
```

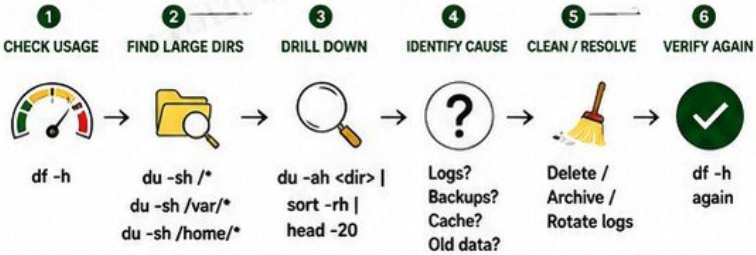
▶ List block devices

```
$ lsblk
NAME   MAJ:MIN  RM   SIZE  RO  TYPE  MOUNTPOINTS
sda      8:0     0   150G   0  disk
 ├─sda1  8:1     0    50G   0  part  /
 └─sda2  8:2     0   100G   0  part  /home
sdb      8:16    0   200G   0  disk
 └─sdb1  8:17    0   200G   0  part  /data
```

▶ Show mounted filesystems

```
$ mount | column -t
/dev/sda1 on  / type  ext4 (rw,relatime)
/dev/sda2 on  /home type ext4 (rw,relatime)
/dev/sdb1 on  /data  type xfs (rw,relatime)
```

9 PRACTICAL: INVESTIGATE WHY A SERVER IS RUNNING OUT OF DISK SPACE



BEST PRACTICE
Monitor disk usage regularly and automate alerts before the disk is full.

WARNING
Never delete files you don't understand. Always verify first.

VERIQTA

LINUX LOGS AND TROUBLESHOOTING

FIND ISSUES FAST. READ LOGS SMART.

1 WHY LINUX LOGS MATTER

- Logs tell you what happened, when it happened and why.
- Essential for troubleshooting, security and auditing.
- Everything is a file in Linux – including logs.

2 /VAR/LOG DIRECTORY

/var/log

- Stores system, service and application logs.
- Different logs for different purposes.
- Text files you can read and search.

3 JOURNALCTL (SYSTEM LOGS)

journalctl

- Reads systemd journal logs.
- Includes boot logs and service logs.
- Powerful filtering options.

```
# View all logs                journalctl
# View logs since boot         journalctl -b
# View logs for a service      journalctl -u nginx
# Follow logs in real-time     journalctl -f
```

4 DMESG (KERNEL RING BUFFER)

- Shows kernel messages.
- Hardware detection, drivers and boot issues.
- Very useful for system startup problems.

```
# Show all kernel messages
dmesg
# Show last 50 kernel messages
dmesg | tail -50
# Live kernel messages
dmesg -w
```

5 TAIL -F (FOLLOW LOGS)

LOG

- Watches a log file in real time.
- Great for live troubleshooting.
- Updates as new lines are written.

```
# Follow a log file
tail -f /var/log/syslog
# Follow service log
tail -f /var/log/nginx/error.log
```

6 SEARCHING LOGS WITH GREP

grep

- Find text, errors or warnings.
- Case-sensitive by default.
- Use -i for ignore case.
- Combine with tail, cat or journalctl.

```
# Search for "error" in syslog
grep "error" /var/log/syslog
# Search ignoring case
grep -i "failed" /var/log/syslog
# Search in real-time log
tail -f /var/log/syslog | grep --line-buffered "error"
```

7 SERVICE LOGS (EXAMPLES)

- Every service writes logs.
- Location depends on the service.
- Check the main log files for issues.

SERVICE	LOG FILE
nginx	/var/log/nginx/error.log
apache2	/var/log/apache2/error.log
sshd	/var/log/secure or /var/log/auth.log
mysql	/var/log/mysql/error.log
docker	journalctl -u docker
systemd services	journalctl -u <service-name>

8 AUTHENTICATION LOGS

- Track login attempts and auth events.
- Important for security monitoring.
- Common files:

PURPOSE	LOG FILE
Auth events	/var/log/auth.log (Debian/Ubuntu)
Auth events	/var/log/secure (RHEL/CentOS/Rocky)
Failed logins	/var/log/faillog
Last logins	/var/log/lastlog

9 READING ERRORS SYSTEMATICALLY

- Read the latest entries first.
- Look for ERROR, FAIL, DENIED, WARN.
- Check timestamps.
- Follow the flow of events.
- Cross-check related logs.

1. Identify the problem.
2. Check the relevant log files.
3. Look for errors or warnings.
4. Understand the cause.
5. Fix the issue.
6. Verify and monitor.

10 PRACTICAL: INVESTIGATE WHY A SERVICE FAILED TO START

1 CHECK SERVICE STATUS
Check if the service failed to start.
`systemctl status <service-name>`

2 VIEW SERVICE LOGS
Look for the reason in the logs.
`journalctl -u <service-name> --no-pager`

3 SEARCH FOR ERRORS
Filter logs for errors or fails.
`journalctl -u <service-name> | grep -i "error\|fail\|denied"`

4 CHECK RELATED LOGS
Check system, auth or app logs.
`tail -n 100 /var/log/syslog /var/log/auth.log`

5 IDENTIFY & FIX
Find root cause and resolve it.

6 VERIFY SERVICE
Restart and confirm service is running.
`systemctl restart <service-name>`
`systemctl status <service-name>`

BEST PRACTICE
Always check logs before restarting or changing configurations.

TIP
Logs are your best friend. The more you read, the faster you solve problems.

WARNING
Never ignore repeated errors. They can cause bigger problems later.

VERIQTA

ENVIRONMENT VARIABLES AND SHELL BASICS

CONTROL YOUR ENVIRONMENT. AUTOMATE YOUR WORK.

1 WHAT THE SHELL DOES

- The shell is a program that reads your commands.
- It interprets and executes them.
- It is your interface to Linux.
- It can run programs, scripts and built-ins.

2 BASH

- Bash (Bourne Again Shell) is the most common shell on Linux.
- Configuration file: ~/.bashrc
- Run: echo $SHELL to see your current shell.

3 ENVIRONMENT VARIABLES

- Variables store information.
- Environment variables are available to the shell and child processes.
- They control how your system behaves.

4 ECHO

Displays text or the value of a variable.

```
$ echo Hello Linux
Hello Linux

$ echo $USER
ubuntu
```

5 ENV

Prints all environment variables.

```
$ env
USER=ubuntu
HOME=/home/ubuntu
PATH=/usr/local/bin:/usr/bin:/bin
SHELL=/bin/bash
...
```

6 EXPORT

Creates or updates an environment variable.

```
$ export MYVAR="DevOps"
$ echo $MYVAR
DevOps
```

★ Use export to make a variable available to child processes.

7 $PATH

Tells the shell where to look for executable commands.

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:\
/usr/sbin:/usr/bin:/sbin:/bin
```

★ When you type a command, Linux searches the directories in $PATH in order.

8 .BASHRC

Runs every time you open a new terminal.

- Put your custom variables, aliases and settings here.
- Location: ~/.bashrc

```
$ nano ~/.bashrc
export DEVOPS="Awesome"
export PATH="$PATH:/opt/mybin"
$ source ~/.bashrc
```

9 SHELL VARIABLES

Variables created by you (not always exported).

```
$ NAME="Veriqta"
$ echo $NAME
Veriqta
```

★ Not exported = not available to child processes.

10 COMMAND SUBSTITUTION

$( ) Run a command inside $() and use its output.

```
$ TODAY=$(date +%Y-%m-%d)
$ echo "Today is $TODAY"
Today is 2024-05-11
```

11 QUOTING BASICS

"Double quotes"
- Expand variables.
  $ echo "Path: $PATH"

'Single quotes'
- Do NOT expand variables.
  $ echo 'Path: $PATH'

`Backticks` (legacy)
- Same as $() but older.
  $ echo `date`

12 USEFUL BUILT-IN COMMANDS

pwd	Print working directory
cd	Change directory
type <cmd>	Show how a command is interpreted
which <cmd>	Show full path of command
alias	List aliases
unalias <name>	Remove alias

13 PRACTICAL: CREATE AN ENVIRONMENT VARIABLE AND UNDERSTAND HOW LINUX LOCATES EXECUTABLE COMMANDS

1 CREATE DIRECTORY

Create a custom bin directory.

```
$ mkdir -p ~/mybin
```

2 CREATE SCRIPT

Create a simple script.

```
$ nano ~/mybin/hello.sh
#!/bin/bash
echo "Hello Veriqta!"
$ chmod +x ~/mybin/hello.sh
```

3 RUN WITH FULL PATH

Run using full path.

```
$ ~/mybin/hello.sh
Hello Veriqta!
```

4 ADD TO PATH

Add ~/mybin to PATH.

```
$ export PATH="$PATH:~/mybin"
$ echo $PATH
...:/home/ubuntu/mybin
```

5 RUN BY COMMAND NAME

Now run by command name.

```
$ hello.sh
Hello Veriqta!
```

6 PERSISTENT PATH

Make it permanent.

```
$ echo 'export
PATH="$PATH:~/mybin"' >> ~/.bashrc
$ source ~/.bashrc
```

VERIFY HOW LINUX LOCATES COMMANDS

```
$ which hello.sh
/home/ubuntu/mybin/hello.sh
```

```
$ type hello.sh
hello.sh is /home/ubuntu/mybin/hello.sh
```

Linux searched the directories in $PATH and found your command!

BEST PRACTICE
Store custom scripts in your own directory and add it to PATH.

TIP
Use double quotes when you want variables to expand, single quotes to keep them literal.

WARNING
Never edit system files unless you know exactly what you are doing.

VERIQTA

YOUR FIRST BASH SCRIPT

AUTOMATE TASKS. SAVE TIME. SOLVE REAL PROBLEMS.

1 WHAT SHELL SCRIPTING SOLVES

- Automate repetitive tasks.
- Combine commands.
- Reduce human error.
- Monitor and report system status.
- Save time and increase consistency.

2 SHEBANG (#!)

- The shebang tells Linux which interpreter to use.
- Always the first line.
- Most common:

```
#!/bin/bash
```

3 VARIABLES

- Store data for later use.
- No spaces around = sign.
- Use $ to access.

```
NAME="Veriqta"
COUNT=10
echo "Hello $NAME"
```

4 USER INPUT

- Accept input from users.
- Use read command.

```
echo -n "Enter your name: "
read USER
echo "Hello $USER"
```

5 IF STATEMENTS

- Make decisions in scripts.
- Use if, elif, else, fi.

```
if [ "$DISK" -gt 90 ]; then
  echo "Disk is full!"
else
  echo "Disk is OK"
fi
```

6 EXIT CODES

- Every command returns an exit status.
- 0 = Success
- Non-zero = Error

```
command
echo $?  # Check last command status
exit 1   # Exit with error code
```

7 SIMPLE LOOPS

- Repeat tasks easily.
- Common loop: for

```
for i in 1 2 3 4 5; do
  echo "Count: $i"
done
```

8 MAKE SCRIPTS EXECUTABLE

- Scripts need execute permission to run.
- Use chmod +x.

```
chmod +x myscript.sh
ls -l myscript.sh
```

9 RUNNING SCRIPTS

- Run using ./script.sh or bash script.sh
- Current directory needs ./ prefix.

```
./myscript.sh
bash myscript.sh
```

10 BASIC ERROR HANDLING

- Check for errors and handle them.
- Use exit codes and conditions.
- Always validate important commands.

```
if [ $? -ne 0 ]; then
  echo "Error: Command failed!"
  exit 1
fi
```

QUICK REFERENCE

TOPIC	EXAMPLE
Shebang	`#!/bin/bash`
Variable	`NAME="Linux"`
User Input	`read INPUT`
If Statement	`if [ "$A" -eq "$B" ]; then`
For Loop	`for i in {1..5}; do echo $i; done`
Make Executable	`chmod +x script.sh`
Run Script	`./script.sh`
Exit with Error	`exit 1`

11 PRACTICAL: BUILD A SIMPLE SERVER HEALTH-CHECK SCRIPT

This script checks disk usage, memory usage, uptime and a service status, then prints a clean report.

SCRIPT: health_check.sh

```
#!/bin/bash

# Health Check Script

DISK=$(df -h / | awk 'NR==2 {print $5}' | tr -d '%')
MEM=$(free | awk '/Mem:/ {printf "%.0f", $3/$2 * 100}')
UPTIME=$(uptime -p)
SERVICE="nginx"

echo "======= SERVER HEALTH CHECK ======="
echo  "Disk Usage       : ${DISK}%"
echo  "Memory Usage     : ${MEM}%"
echo  "Uptime           : ${UPTIME}"
if systemctl is-active --quiet $SERVICE; then
  echo "${SERVICE} Status : RUNNING (OK)"
  exit 0
else
  echo "${SERVICE} Status : NOT RUNNING (ERROR)"
  exit 1
fi
```

MAKE IT EXECUTABLE

```
chmod +x health_check.sh
ls -l health_check.sh
```

RUN THE SCRIPT

```
./health_check.sh
```

SAMPLE OUTPUT (SUCCESS)

```
====== SERVER HEALTH CHECK =======
Disk Usage       : 42%
Memory Usage     : 67%
Uptime           : up 3 days, 2 hours
nginx Status     : RUNNING (OK)
```

SAMPLE OUTPUT (SERVICE DOWN)

```
====== SERVER HEALTH CHECK =======
Disk Usage       : 42%
Memory Usage     : 67%
Uptime           : up 3 days, 2 hours
nginx Status     : NOT RUNNING (ERROR)
```

HOW IT WORKS

1. Get disk usage of root (/)
2. Get memory usage percentage
3. Get system uptime
4. Check if service is running
5. Print report and return proper exit code

BEST PRACTICE
Keep scripts simple, readable and well-commented.

TIP
Always test your script. Validate every command.

WARNING
A wrong script can harm your system. Always review before running.

VERIQTA

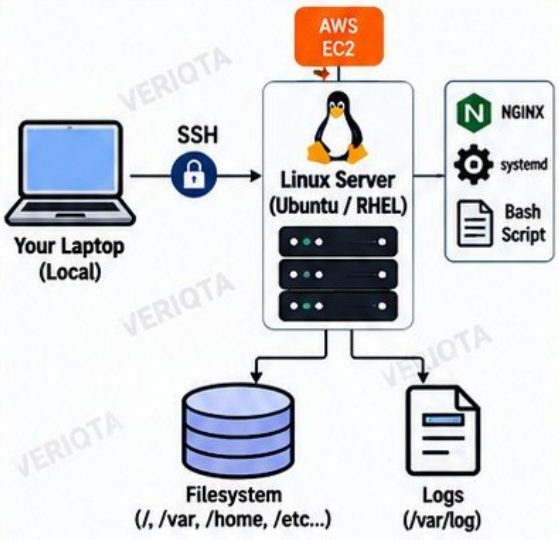
BEGINNER LINUX SERVER PROJECT

BUILD, CONFIGURE AND TROUBLESHOOT A REAL LINUX SERVER

1 PROJECT TASKS

- [ ] 1 Connect to the server through SSH
- [ ] 2 Inspect the operating system
- [ ] 3 Create users and groups
- [ ] 4 Configure permissions
- [ ] 5 Create application directories
- [ ] 6 Install a web server
- [ ] 7 Start and enable the service
- [ ] 8 Verify the listening port
- [ ] 9 Test the service with curl
- [ ] 10 Create and move files
- [ ] 11 Inspect processes
- [ ] 12 Check disk usage
- [ ] 13 Inspect system logs
- [ ] 14 Diagnose a deliberately failed service
- [ ] 15 Write a basic health-check script

2 PROJECT ARCHITECTURE



3 SAMPLE TERMINAL SESSION

```
$ ssh ubuntu@YOUR_SERVER_IP
$ hostnamectl
  Static hostname: veriqta-server
  Operating System: Ubuntu 22.04 LTS

$ sudo useradd devops
$ sudo usermod -aG sudo devops
$ sudo mkdir -p /opt/myapp
$ sudo chown -R devops:devops /opt/myapp

$ sudo apt update
$ sudo apt install nginx -y

$ sudo systemctl start nginx
$ sudo systemctl enable nginx
$ systemctl status nginx
    Active: active (running)

$ ss -tuln | grep :80
LISTEN    0      511    0.0.0.0:80

$ curl -I localhost
HTTP/1.1 200 OK
```

4 USEFUL COMMANDS RECAP

TASK	COMMAND(S)
Check OS	uname -a, hostnamectl
Disk usage	df -h, du -sh *
Processes	ps aux, top
Services	systemctl status <svc>
Logs	tail -f /var/log/<file>
Network/Ports	ss -tuln, curl, ping
Search files	find, grep, locate
Permissions	chmod, chown, id

5 SERVER HEALTH-CHECK SCRIPT

health_check.sh

```
#!/bin/bash
# Simple Linux server health check

echo "===== LINUX SERVER HEALTH CHECK ====="
echo "Date: $(date)"
echo "Hostname: $(hostname)"

echo -e "\n[1] DISK USAGE"
df -h | grep '^/dev/'

echo -e "\n[2] MEMORY USAGE"
free -h

echo -e "\n[3] UPTIME"
uptime

echo -e "\n[4] SERVICE STATUS (nginx)"
systemctl is-active nginx && echo "nginx is
RUNNING" || echo "nginx is NOT RUNNING"
```

✔ chmod +x health_check.sh ✔ ./health_check.sh

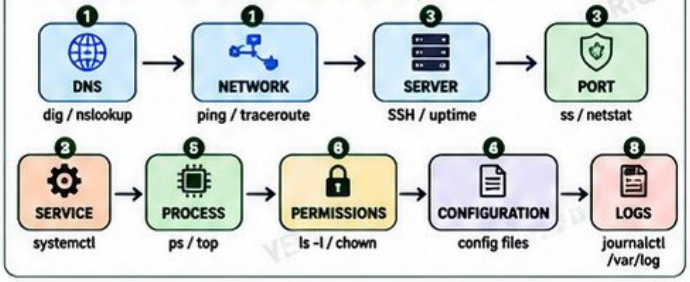
6 SAMPLE SCRIPT OUTPUT

```
===== LINUX SERVER HEALTH CHECK =====
Date: Sun Aug 24 10:20:15 UTC 2026
Hostname: veriqta-server

[1] DISK USAGE
Filesystem   Size Used Avail Use% Mounted on
/dev/xvda1    50G  12G  36G  25% /

[2] MEMORY USAGE
               total    used    free
Mem:           1.9G    820M    1.1G
Swap:          1.0G      0B    1.0G

[3] UPTIME
 up 3 days, 2 hours, 14 min

[4] SERVICE STATUS (nginx)
nginx is RUNNING ✅
```

7 FINAL TROUBLESHOOTING CHALLENGE

⚠ A web application is unavailable.

Follow the investigation path step by step:



8 TROUBLESHOOTING WORKFLOW

1. Reproduce the problem.
2. Check connectivity (DNS + Network).
3. Verify the server is reachable.
4. Check if the port is listening.
5. Confirm the service is running.
6. Inspect process and resource usage.
7. Check permissions and config files.
8. Read system and service logs.
9. Fix the root cause.
10. Verify and monitor.

PRO TIP
Logs + system status + network checks = Faster root cause.

9 KEY TAKEAWAYS

- Comfortable using a real Linux server.
- Automate common tasks with shell scripts.
- Monitor system health.
- Troubleshoot like a DevOps engineer.
- Build a strong foundation for Cloud, Kubernetes, SRE and Platform Engineering.

CONGRATULATIONS!
You have completed Linux for Complete Beginners! 🚀

VERIQTA

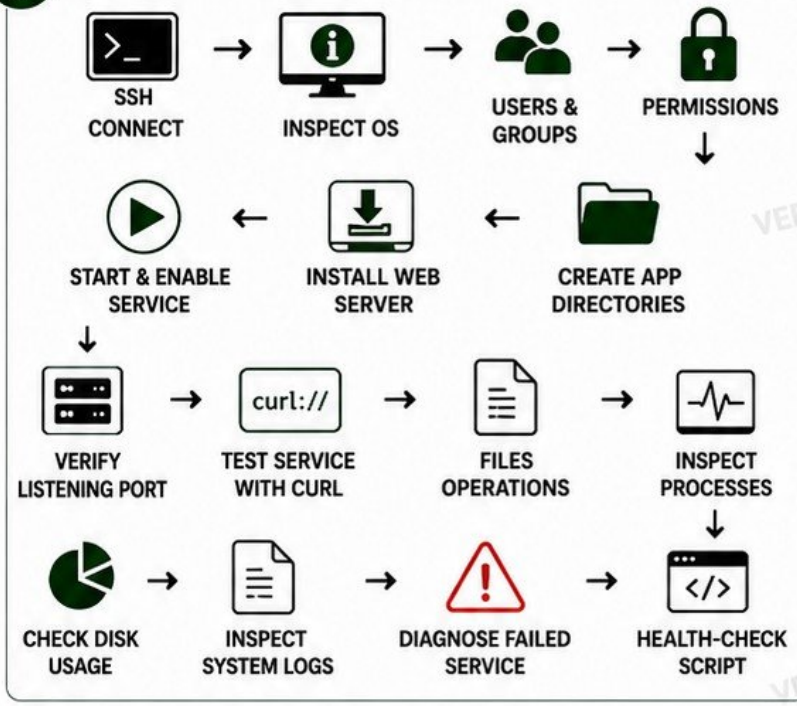
BEGINNER LINUX SERVER PROJECT

BUILD AND TROUBLESHOOT A LINUX SERVER FROM START TO FINISH.

PROJECT TASKS

1. CONNECT TO THE SERVER THROUGH SSH
2. INSPECT THE OPERATING SYSTEM
3. CREATE USERS AND GROUPS
4. CONFIGURE PERMISSIONS
5. CREATE APPLICATION DIRECTORIES
6. INSTALL A WEB SERVER
7. START AND ENABLE THE SERVICE
8. VERIFY THE LISTENING PORT
9. TEST THE SERVICE WITH CURL
10. CREATE AND MOVE FILES
11. INSPECT PROCESSES
12. CHECK DISK USAGE
13. INSPECT SYSTEM LOGS
14. DIAGNOSE A DELIBERATELY FAILED SERVICE
15. WRITE A BASIC HEALTH-CHECK SCRIPT

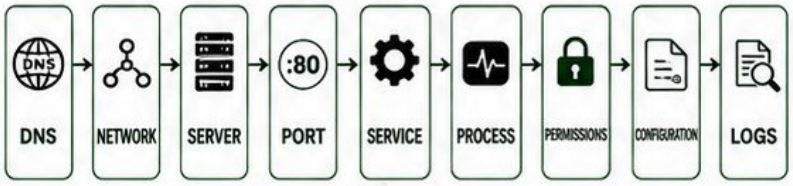
PROJECT FLOW



FINAL TROUBLESHOOTING CHALLENGE

A WEB APPLICATION IS UNAVAILABLE.

THE LEARNER MUST DETERMINE WHETHER THE PROBLEM COMES FROM:



SUCCESS CRITERIA

YOU CAN BUILD A WORKING LINUX SERVER.

- YOU CAN VERIFY SERVICES AND PORTS.
- YOU CAN DIAGNOSE AND FIX COMMON ISSUES.
- YOU UNDERSTAND HOW LINUX COMPONENTS WORK TOGETHER.
- YOU CAN TROUBLESHOOT LOGICALLY AND CONFIDENTLY.

BEST PRACTICES

- ALWAYS WORK WITH THE LEAST PRIVILEGE.
- KEEP SYSTEMS UPDATED.
- MONITOR SERVICES AND LOGS REGULARLY.
- AUTOMATE CHECKS AND ALERTS.
- DOCUMENT WHAT YOU BUILD AND LEARN.

REMEMBER: IN LINUX, EVERYTHING IS A FILE, LOGS TELL THE STORY, AND A CALM MIND SOLVES THE PROBLEM.

PRACTICE. EXPLORE. BREAK. FIX. REPEAT.